מספר התלמיד הנבחן
רשום את כל תשע הספרות

הדבק כאן את
מדבקת הנבחן

האוניברסיטה
הפתוחה

י"ט בתמוז תשע"ח

מס' שאלון - 521

2

ביולי 2018

סמסטר 2018ב

מס' מועד 84

20905 / 4

שאלון בחינת גמר

20905 - שפות תכנות

משך בחינה: 3 שעות

בשאלון זה **10** עמודים

מבנה הבחינה:

בבחינה ארבע שאלות.

עליכם לענות על כל השאלות.

משקל השאלות מפורט בגוף השאלות.

שימו לב:

את תשובותיכם רשמו אך ורק במחברת הבחינה.

כתבו בכתב יד ברור ומסודר.

חומר עזר:
ספר הקורס: essentials of programming languages
מדריך הלמידה. מותרות הערות בכתב יד, ע"ג הספרים.
אין להכניס חומר מודפס או כל חומר אחר מכל סוג שהוא.

בהצלחה !!!

אינכם חייבים
להחזיר את השאלון לאוניברסיטה הפתוחה

שאלה 1 (25 נקודות)

הרחיבו את שפת LET (שפת "ויהי") כך שביטוי if יבוצע על פי הדקדוק הבא:

להלן הדקדוק המגדיר את התחביר החדש עבור ביטוי if :

***Expression* ::= if *Expression* then { {*Expression*}$^{+(;)}$}**
else { {*Expression*}$^{+(;)}$}

if-exp (bool exps1 exps2)

כאשר,

- bool הוא ביטוי בוליאני.
- exps1 הם אוסף ביטויים המופרדים בתו ; שאמורים להתבצע כאשר ערכו של הביטוי הבוליאני הוא true. הערך המוחזר במקרה זה יהיה ערכו של הביטוי האחרון שבוצע.
- exps2 הם אוסף ביטויים המופרדים בתו ; שאמורים להתבצע כאשר ערכו של הביטוי הבוליאני הוא false. הערך המוחזר במקרה זה יהיה ערכו של הביטוי האחרון שבוצע.

להלן דוגמא להמחשת השימוש בביטוי החדש:

```
let x= 7
in
  if zero? (x) then { -(x,4); -(x,9); x}
                 else { if –(x,3) then {3} else {6} ; -4 }
```

בדוגמה זו, ערכו של x אינו 0 ולכן יבוצע בלוק ה-else ויוחזר ערכו. בבלוק ה-else מופיעים 2 ביטויים לביצוע. לאחר ביצועם, יוחזר ערכו של האחרון שבהם, במקרה זה (num-val -4)

א. (7 נקודות)

להלן נתונים ה-scanner, וה-parser של שפת "ויהי". כתבו מהו השינוי הדרוש בדקדוק של ביטוי if כך שיתאים להגדרת הדקדוק החדש. **<u>את תשובותיכם כתבו אך ורק במחברת הבחינה.</u>**

ה-scanner:

```
(define the-lexical-spec
  '((whitespace (whitespace) skip)
    (comment ("%" (arbno (not #\newline))) skip)
    (identifier (letter (arbno (or letter digit "_" "-" "?")))  symbol)
    (number (digit (arbno digit)) number)
    (number ("-" digit (arbno digit)) number)
  ))
```

ה-parser:

```
(define the-grammar
  '((program (expression) a-program)
    (expression (number) const-exp)
    (expression ("-" "(" expression "," expression ")") diff-exp)
    (expression ("zero?" "(" expression ")") zero?-exp)
    (expression ("if" expression "then" expression "else" expression) if-exp)
    (expression (identifier) var-exp)
    (expression ("let" identifier "=" expression "in" expression) let-exp)
   ))
```

ב. (18 נקודות)

להלן הפרוצדורה *value-of* של שפת **"ויהי"** כפי שמופיעה בספר הלימוד. יש לעדכנה כך שתחשב את ביטוי if על פי הדקדוק החדש. **<u>את התשובה כיתבו במחברת הבחינה</u>**.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "ויהי".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

הפרוצדורה *value-of* (כפי שמופיעה בספר בעמוד 71):

```
;;value-of :  Exp x Env ----> ExpVal
 (define value-of
  (lambda (exp env)
   (cases expression exp
    (const-exp (num) (num-val num))
    (var-exp (var) (apply-env env var))
    (diff-exp (exp1 exp2)
       (let ((val1 (value-of exp1 env))
             (val2 (value-of exp2 env)))
         (let ((num1 (expval->num val1))
               (num2 (expval->num val2)))
          (num-val (- num1 num2)))))
    (zero?-exp (exp1)
      (let ((val1 (value-of exp1 env)))
        (let ((num1 (expval->num val1)))
          (if (zero? num1) (bool-val #t)  (bool-val #f)))))
    (if-exp (exp1 exp2 exp3)
      (let ((val1 (value-of exp1 env)))
        (if (expval->bool val1) (value-of exp2 env) (value-of exp3 env))))
    (let-exp (var exp1 body)
      (let ((val1 (value-of exp1 env)))
         (value-of body (extend-env var val1 env))))
    )))
```

שאלה 2 (25 נקודות)

שאלה זו עוסקת בשדרוג של שפת "וישגור" (שפת PROC) המתוארת בספר הלימוד בפרק 3 בעמודים 74-81. שפה זו מאפשרת הגדרה והפעלה של פרוצדורות לא רקורסיביות.

ברצוננו להרחיב את השפה עם ביטוי חדש שתחבירו נתון על-ידי הדקדוק הבא:

Expression ::= **mapsum** ***Expression*** **[{** ***Expression*** **}**$^{*(;)}$**]**

```
mapsum-exp (prc exps)
```

כאשר,

- prc היא ביטוי המתאר פרוצדורה המקבלת מספר ומחזירה מספר
- exps אוסף של 0 או יותר ביטויים שתוצאתן היא מספר. הביטוי החדש מפעיל את הפרוצדורה prc על כל אחד מהביטויים באוסף exps ומחזיר את סכום התוצאות שהתקבלו מכל ההפעלות. במקרה שהאוסף exps ריק יוחזר סכום שערכו 0.

להלן דוגמה להמחשת השימוש בביטוי החדש:

```
let p= proc (x) –(x,1)
in
  mapsum p [6; -(9,2); 2]
```

בדוגמה זו, הפרוצדורה p מופעלת על כל אחד מ-3 הביטויים באוסף, ומחזירה עבור כל אחד מהם את ערכו פחות 1. ולכן התוצאה הסופית תהיה הסכום של כל הערכים שהוחזרו מהפעלת p על כל אחד מהביטויים באוסף. במקרה זה, נקבל (num-val 12)

א. (7 נקודות)

להלן נתונים ה-**scanner** וה-**parser** של שפת **"וישגור"**. עליכם להשלים את החסר בריבוע המסומן.**<u>את התשובה כיתבו במחברת הבחינה</u>**.

ה-scanner:

```
(define the-lexical-spec
'((whitespace (whitespace) skip)
(comment ("%" (arbno (not #\newline))) skip)
  (identifier(letter (arbno (or letter digit "_" "-" "?")))
           symbol)
  (number (digit (arbno digit)) number)
(number ("-" digit (arbno digit)) number)
))
```

ה-parser:

```
(define the-grammar
    '((program (expression) a-program)

      (expression (number) const-exp)
      (expression
        ("-" "(" expression "," expression ")")
diff-exp)

      (expression
       ("zero?" "(" expression ")")
zero?-exp)

      (expression
       ("if" expression "then" expression "else" expression)
if-exp)

      (expression (identifier) var-exp)

      (expression
       ("let" identifier "=" expression "in" expression)
let-exp)

      (expression
       ("proc" "(" identifier ")" expression)
proc-exp)

      (expression
       ("(" expression expression ")")
call-exp)
```

```
(expression
```

את התשובה יש לכתוב במחברת הבחינה

```
mapsum-exp)))
```

ב. (18 נקודות)

להלן הפרוצדורה *value-of* של שפת **"וישגור"** . ולאחריה פרוצדורת העזר apply-procedure כפי שמופיעה בספר הלימוד בעמוד 79. יש להשלים את החסר בפרוצדורה value-of. **<u>את התשובה כיתבו במחברת הבחינה</u>**.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "וישגור".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

;;value-of : Exp x Env→ExpVal

```
(define value-of
(lambda (exp env)
(cases expression exp
(const-exp (num) (num-val num))
(var-exp (var) (apply-env env var))
(diff-exp (exp1 exp2)
  (let ((val1 (value-of exp1 env))
(val2 (value-of exp2 env)))
(let ((num1 (expval->num val1))
   (num2 (expval->num val2)))
(num-val(- num1 num2)))))
(zero?-exp (exp1)
(let ((val1 (value-of exp1 env)))
(let ((num1 (expval->num val1)))
(if (zero? num1)(bool-val #t)(bool-val #f)))))
(if-exp (exp1 exp2 exp3)
 (let ((val1 (value-of exp1 env)))
 (if (expval->bool val1)
(value-of exp2 env)
(value-of exp3 env))))
       (let-exp (var exp1 body)
 (let ((val1 (value-of exp1 env)))
 (value-of body(extend-env var val1 env))))

     (proc-exp (var body)
          (proc-val (procedure var body env)))

(call-exp (rator rand)
          (let ((proc (expval->proc (value-of rator env)))
                (arg (value-of rand env)))
            (apply-procedure proc arg)))
```

```
(mapsum-exp  (________________________________)
```

את התשובה יש לכתוב במחברת הבחינה

```
                                                    ))))
```

```
;; apply-procedure : Proc * ExpVal ->ExpVal
;; Page: 79
  (define apply-procedure
    (lambda (proc1 val)
      (cases proc proc1
        (procedure (var body saved-env)
          (value-of body (extend-env var val saved-env))))))
```

שאלה 3 (25 נקודות)

בספר הלימוד בעמודים 113-120 מתוארת שפת **"וירמוז"**(IMPLICIT-REFS).

ברצוננו להרחיב שפה זו עם ביטוי חדש בשם do-let

להלן הדקדוק המגדיר את הביטוי החדש שברצוננו להוסיף:

Expression ::= do-let (identifier *Expression Expression*) (*Expression Expression*)

do-let-exp (tmpid init nxt bool retval)

לביטוי **do-let-exp** המרכיבים הבאים:

- **tmpid** – משתנה זמני שמוכר רק במסגרת פעולת הביטוי. כלומר בסיום חישוב הביטוי חוזרים לעבוד עם הסביבה שהייתה טרם ביצוע ביטוי ה-do-let.
- **init** – ערך התחלתי שיינתן למשתנה הזמני בתחילת ביצוע הביטוי.
- **nxt** – ביטוי שערכו יינתן למשתנה הזמני בכל פעם שערכו של ביטוי bool הוא false
- **bool** – ביטוי בוליאני שכל עוד ערכו **false** כל הביטוי יבוצע מחדש.
- **retval** – ביטוי שערכו יוחזר כתשובה סופית כאשר התנאי הבוליאני יהיה true

שלבי הביצוע של ביטוי do-let הם:
בתחילת חישוב הביטוי יוגדר משתנה זמני לצורך חישוב הביטוי, המשתנה המצוין ע"י tmpid. משתנה זה יאותחל בערכו של הביטוי init. לאחר מכן יחושב הביטוי הבוליאני המצוין ע"י bool, אם ערכו false, המשתנה tmpid יקבל כעת את ערכו של הביטוי nxt. ואז שוב ייבדק ערכו של התנאי הבוליאני bool וכל עוד הוא false, tmpid ימשיך לקבל את ערכו nxt. כאשר ערכו של הביטוי הבוליאני הוא true, יש להפסיק לבצע את ביטוי do-let וערכו של ביטוי retval יוחזר כתשובה סופית.
שימו לב, המשתנה הזמני המצויין ע"י tmpid הוא משתנה חדש שיש ליצור באופן זמני. כמו כן, כל אחד מהביטויים המצויינים בביטוי do-let צריך להיות מחושב מחדש בכל פעם שהוא נדרש.

להלן דוגמא:

```
let x=7
in let y= do-let (x 10 +(x,1)) (zero? -(x,15) x)
   in -(y,x)
```

בדוגמה זו, מוגדר משתנה x עם ערך 7. לאחר מכן, מוגדר משתנה y המקבל את ערכו של ביטוי do-let, ביטוי ה-do-let מגדיר משתנה x (חדש וזמני רק לצורך ה- do-let) משתנה זה מאותחל ב-10, וכל עוד x שונה מ- 15 ערכו גדל ב-1. בהגיע x ל- 15 מוחזר ערכו של x, שהוא כרגע 15. ולכן משתנה y מקבל ערך של 15. כעת המשתנה x אליו תבוצע פנייה במסגרת גוף let -v השני הוא ה-x שערכו 7. ולכן ערכו של כל הביטוי הוא 8= 7-15
ולכן יוחזר (num-val 8)

א. (7 נקודות)

להלן נתונים ה-**scanner** וה-**parser** של שפת "**וירמוז**". עליכם להשלים את החסר בריבוע המסומן.**<u>את התשובה כיתבו במחברת הבחינה</u>**.

ה-scanner:

```
(define the-lexical-spec
'((whitespace (whitespace) skip)
(comment ("%" (arbno (not #\newline))) skip)
  (identifier(letter (arbno (or letter digit "_" "-" "?")))
           symbol)
  (number (digit (arbno digit)) number)
(number ("-" digit (arbno digit)) number)
))
```

ה-parser:

```
(define the-grammar
'((program (expression) a-program)
  (expression (number) const-exp)
  (expression ("-" "(" expression "," expression ")")
    diff-exp)
  (expression ("zero?" "(" expression ")") zero?-exp)
  (expression
   ("if" expression "then" expression "else" expression)
     if-exp)
  (expression (identifier) var-exp)
  (expression ("let" identifier "=" expression "in" expression)
     let-exp)
  (expression ("proc" "(" identifier ")" expression)
     proc-exp)
  (expression ("(" expression expression ")") call-exp)
  (expression
        ("letrec"
          (arbno identifier "(" identifier ")" "=" expression)
           "in" expression)
letrec-exp) (expression ("set" identifier "=" expression)
    assign-exp)
(expression (
```

את התשובה כיתבו במחברת הבחינה

```
)
  do-let-exp)))
```

ב. (18 נקודות)

להלן הפרוצדורות *value-of* ו- *apply-procedure* של שפת **"וירמוז"**.

הסבירו במדויק מה הם השינויים והתוספות הנדרשים לשילוב הביטוי החדש בשפה זו.

<u>את התשובה כיתבו במחברת הבחינה</u>.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "וירמוז".
- המנשק לעבודה עם החסן (store) הוא אותו מנשק כפי שהוגדר עבור שפת "וירמוז".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

```
;; value-of : Exp * Env ->ExpVal
;; Page: 118, 119
  (define value-of
    (lambda (exp env)
      (cases expression exp
        (const-exp (num) (num-val num))

       (var-exp (var) (deref (apply-env env var)))

        (diff-exp (exp1 exp2)
          (let ((val1 (value-of exp1 env))
                (val2 (value-of exp2 env)))
            (let ((num1 (expval->num val1))
                  (num2 (expval->num val2)))
              (num-val
                (- num1 num2)))))

        (zero?-exp (exp1)
          (let ((val1 (value-of exp1 env)))
            (let ((num1 (expval->num val1)))
              (if (zero? num1)
                (bool-val #t)
                (bool-val #f)))))

        (if-exp (exp1 exp2 exp3)
          (let ((val1 (value-of exp1 env)))
            (if (expval->bool val1)
              (value-of exp2 env)
              (value-of exp3 env))))

        (let-exp (var exp1 body)
          (let ((v1 (value-of exp1 env)))
            (value-of body
              (extend-env var (newref v1) env))))

        (proc-exp (var body)
          (proc-val (procedure var body env)))

        (call-exp (rator rand)
          (let ((proc (expval->proc (value-of rator env)))
                (arg (value-of rand env)))
            (apply-procedure proc arg)))
```

```
      (letrec-exp (p-names b-vars p-bodies letrec-body)
          (value-of letrec-body
            (extend-env-rec* p-names b-vars p-bodies env)))

        (assign-exp (var exp1)
          (begin
            (setref!
              (apply-env env var)
              (value-of exp1 env))
            (num-val 27))))))


(define apply-procedure
      (lambda (proc1 val)
         (cases proc proc1
            (procedure (var body saved-env)
               (value-of body
                  (extend-env var (newref val) saved-env))))))
```

שאלה 4 (25 נקודות)

להלן נתונה תוכנית בשפת "ויקיש" (INFERRED):

```
let  p = proc (w:?) (w 7)
in
  proc (m:?)
    (p m)
```

ברצוננו לקבוע מהו טיפוס התוכנית (אם קיים).לשם כך, עליכם להשתמש באלגוריתם שנלמד בפרק 7 להקשת הטיפוס.

א. (10 נקודות)

הקצו לביטוי הנתון בשאלה ולתתי הביטויים שלו משתני-טיפוס (Type Variables) מתאימים, וחברו משוואות מתאימות לביטוי הנתון ולתתי הביטויים שלו.

ב. (15 נקודות)

פתרו את המשוואות שהרכבתם בסעיף א' תוך שימוש ב- substitution ו- unification בדומה למתואר בספר בעמודים 252-258. בסיום פתרון המשוואות רשמו מהו הטיפוס שאותו הקשתם עבור התוכנית הנתונה.

ב ה צ ל ח ה !